I. Approach

Module 1 - 0/1 Knapsack via Dynamic Programming: Determines the optimal combination of relief goods to pack into a single bag given a weight capacity limit.

Module 2 - Greedy Priority Assignment: Ranks families by a computed priority score and assigns bags in descending priority order until supply is exhausted.

II. Pseudocode

Knapsack

Treat every weight as tenths of a kilogram (0.1 kg per step), so all weights and capacity are whole numbers.

Build the dynamic programming table
Given: a list of relief items, maximum bag capacity in kilograms
Produce: the filled table, the capacity limit in scaled units, each item's scaled weight, and how many items there are

1. Convert the bag's capacity in kilograms into scaled weight units.
2. For each relief item, convert its weight in kilograms into scaled units and remember it.
3. For each relief item, remember its benefit value.
4. Count how many relief items there are.
5. Create a table with one extra row and one extra column beyond those counts; set every cell to zero.
6. For each item, from the first through the last:
    a. Note this item's scaled weight and its benefit.
    b. For each possible remaining bag capacity, from empty up to full:
        i. Assume this item is not packed: copy the best benefit from the previous row at the same capacity.
        ii. If this item fits at this capacity:
            1. Compute the benefit if this item is packed: this item's benefit plus the best benefit from the previous row for whatever capacity is left after taking this item's weight.
            2. If packing this item gives a higher benefit than skipping it, store that higher value in the table.
7. Return the table, the scaled capacity limit, the list of scaled weights, and the item count.

Optimize the bag (choose which items to pack)
Given: a list of relief items, maximum bag capacity in kilograms
Produce: which items to pack, total weight in kilograms, total benefit

1. Build the dynamic programming table using the steps above.

2. Start with an empty list of chosen items and pretend the bag still has full capacity (in scaled units).
3. Go through items from last to first:
    a. If the table value with this item at the current capacity is different from the value without this item (the row above), then this item was chosen in an optimal packing:
        i. Add this item to the chosen list.
        ii. Reduce the remaining capacity by this item's scaled weight.
4. Add up the real weights in kilograms of all chosen items; round to two decimal places, that is total weight.
5. Read the benefit in the table for all items and full capacity, that is total benefit.
6. Return the chosen items, total weight, and total benefit.

Get the table for the screen (display only)
Given: a list of relief items, maximum bag capacity in kilograms
Produce: the table and information needed to label it on screen

1. Build the dynamic programming table the same way as above.
2. Return that table together with the item list, the scaled capacity limit, and the weight scale (tenths of a kilogram).

Priority
Compute scores
Input: a list of families
Output: the same list of families with updated scores

1. For each family in the list:
    a. Compute the family's formula-based priority score.
    b. Copy that formula score into the "final score" used for ranking.
2. Return the list of families.

Rank families
Input: a list of families (with final scores already computed)
Output: a new list of families sorted in priority order

1. Sort the families using the following priority rules:
    a. Families with higher final score come first.
    b. If final scores are tied, the larger family size comes first.
    c. If both final score and size are tied, the earlier registration order comes first.
2. Return the sorted list.

Assign bags
Input:

- a list of families
- the optimized bag contents (list of relief items)
- the optimized bag's total weight
- the optimized bag's total benefit
- the number of bags available (supply)

Output: an operation summary containing assignments and totals

1. Compute scores for all families.
2. Rank all families using the ranking procedure.
3. Create an empty list of assignment results.
4. Set "bags left" equal to the supply value.
5. For each family in the ranked list:
    a. Decide whether the family will be served:
        i. The family is served if there is at least one bag left.
    b. If the family is served:
        i. Decrease "bags left" by one.
        ii. Record an assignment where the family receives:
            1. the optimized bag contents,
            2. the optimized bag weight,
            3. the optimized bag benefit,
            4. and a served status of true.
    c. If the family is not served:
        i. Record an assignment where the family receives:
            1. an empty bag contents list,
            2. a bag weight of zero,
            3. a bag benefit of zero,
            4. and a served status of false.
6. Count how many families were served by counting assignment results marked served.
7. Compute total benefit delivered by summing the bag benefit for all served assignments.
8. Create and return a summary containing:
    - total number of families,
    - number served,
    - number unserved,
    - total benefit delivered,
    - the optimized bag details,
    - and the full list of assignment results.

III. Analysis of Time Complexity

Module 1 - 0/1 Knapsack DP

n = Number of relief item types

W = Capacity in DP units = capacity (kg) × 10
F = Number of registered families

Best Case: Ω(nW)
Average Case: O(nW)
Worst Case: Θ(nW)

Module 2 - Greedy priority assignment

Best Case: Ω(FlogF)
Average Case: O(FlogF)
Worst Case: Θ(FlogF)

Knapsack: Best, average, and worst time are all Θ(nW) because we always fill the full DP table; space is also Θ(nW). Greedy assignment: Best, average, and worst time are Θ(F log F) because of sorting; space is Θ(F). Together: Θ(nW + F log F) time. In our barangay scenario, n and W are small constants, so family ranking drives cost as the registry grows.

IV. Correctness of Program

```
PS C:\Users\sdtal\Documents\CMSC142_finalproj> python checker.py full
[PASS] knapsack_case
[PASS] priority_case
.........................................
----------------------------------------------------------------------
Ran 42 tests in 0.027s

OK
[PASS] unit_tests

Summary: PASS=3, FAIL=0
Ran for: 0.0981 seconds
```